TrustLoad: Product Requirements Document

PayPal AI Hackathon 2026 · Draft v1 · Nov 6, 2026 · Submission deadline: Nov 12, 2026

Pool money. Buy together. Everyone sees every naira.

1. Overview

The problem. Across Nigeria and much of Africa, groups pool money and trust one person to hold and spend it, with no proof of where it went. The clearest case is the pooled buying trip: vendors send one person abroad (China, Dubai, Turkey, the UK, or any other country) to buy goods for everyone. The whole arrangement runs on trust in that one traveler. Contributors cannot see how their money is spent, splitting the goods causes arguments, and there is little recourse if the traveler buys the wrong things, overspends, or disappears.

The solution. TrustLoad is a web app that makes group money transparent and safe, starting with pooled buying trips to any country. Vendors fund a group pot through PayPal before the trip. The traveler requests staged releases of money against supplier invoices. AI reads and verifies each invoice against the group's combined order. After the trip, AI reads the final receipts and splits goods and shared costs fairly per vendor. Everyone sees one live ledger.

Why PayPal and AI are both essential. PayPal collects contributions, releases funds in stages, and refunds leftovers. AI reads messy, multilingual invoices and receipts, checks them against the order, and calculates each vendor's share. Without AI, someone would have to verify every receipt by hand, which is the exact trust problem we are solving.

Cross-border by design. The traveler can buy in any country, so invoices arrive in different currencies and languages. The group has one base currency, and every invoice is converted at a recorded exchange rate.

Where it extends. The same engine fits any group where many people fund and one person spends: community project funds (a borehole, a transformer), event and family contributions, and trade association dues. Only the buying trip is built for the hackathon. The rest are pitched as extensions.

2. Goals and non-goals

Goals

- Show a complete, working trip lifecycle: fund, request, AI check, approve, release, receipts, AI split, settle.
- Make every naira traceable to a PayPal sandbox transaction ID.
- Keep the AI useful but never in control of money: it recommends, humans approve.

Non-goals (for the hackathon)

- Multiple simultaneous groups at scale, notifications, chat, or in-app messaging.
- Real-money production payouts to Nigerian bank accounts or customs and logistics tracking.
- Reputation scores for travelers.

3. Users

- **Vendor (contributor).** A small trader who wants goods from the trip. Wants to know where their money is and get exactly their share.
- **Traveler.** The person who goes to market. Wants a smooth way to get funds released and prove they spent them properly.

4. End-to-end user flow

1. A group is created with a traveler, a list of vendor members, and each vendor's order (items, quantities, budget).
2. Each vendor pays their contribution through PayPal. The pot balance updates on payment confirmation.
3. The traveler submits a **release request** (amount plus supplier invoice upload).
4. AI extracts the invoice and checks it against the group's combined order. Mismatches are flagged.
5. Vendors review the AI summary and approve. When the approval threshold is met, PayPal pays out that portion to the traveler.
6. Steps 3 to 5 repeat for each supplier.
7. After the trip, the traveler uploads final receipts. AI allocates goods and shared costs per vendor.
8. Leftover or overpaid amounts are refunded through PayPal. The group closes with a final ledger.

5. Functional requirements

P0 (must ship)

- **FR1 Group and order setup.** Create a group, add members, record each member's order lines and agreed contribution.
- **FR2 Contributions.** Each vendor pays through PayPal. A webhook confirms payment and updates the pot. Handle each webhook idempotently.
- **FR3 Release request.** Traveler submits an amount and uploads an invoice (image or PDF).
- **FR4 AI invoice verification.** Extract line items, quantities, unit prices, currency and total. Compare against the combined order. Output a match report with flags (see section 6).
- **FR5 Approval and payout.** Members approve or reject. When the threshold is reached, trigger a PayPal payout to the traveler. Record the transaction ID.
- **FR6 Final receipts and allocation.** Traveler uploads final receipts. AI assigns line items to vendors and splits shared costs.
- **FR7 Settlement.** Calculate each vendor's final amount owed or refunded and execute refunds through PayPal.
- **FR8 Ledger.** One view showing money in, released, spent, and owed per vendor, with PayPal transaction IDs.

P1 (if time allows)

- Email or WhatsApp-style share link for the ledger.
- Editable AI results (human correction before approval).
- Traveler can attach photos of goods to each release.

6. AI requirements

Invoice and receipt extraction. Input is a photo or PDF, possibly blurry, handwritten, or in Chinese, Arabic, Turkish, English or mixed languages. Output is structured JSON: supplier, date, currency, line items (description, quantity, unit price, line total), and grand total.

Verification checks.

- Line items match items on the group's combined order.
- Quantities do not exceed what was ordered.
- Line totals add up to the invoice total (arithmetic check).
- Invoice total does not exceed the amount requested in the release.

- Unit prices are not wildly above prices seen earlier in the same trip.

Cost allocation. Goods are assigned to the vendor who ordered them. Shared costs (shipping, transport, fees) are split proportionally to each vendor's goods value by default. The rule is configurable per group.

Safety rules.

- AI output is advisory. No payout happens without human approval.
- Each extracted field carries a confidence level. Low-confidence fields are highlighted for manual review.
- Every AI result is stored with the source file for audit.

Language and currency handling. Invoices can be in any language. The AI returns the original currency and amounts, translated item names, and the source language. The conversion to the base currency is calculated in code from the stored exchange rate, not by the model.

7. PayPal integration

- **Contributions:** Orders API for each vendor payment, with a webhook for payment completion.
- **Staged release:** Payouts API from the platform's sandbox business account to the traveler.
- **Refunds:** Refund API against original contributions, or payouts back, depending on what the sandbox supports.
- **Reference:** Store PayPal order IDs, capture IDs and payout batch IDs against every ledger entry.

To verify in the sandbox on day one: whether Payouts is enabled on the sandbox business account, how platform-held funds across several payments should be modelled, and how refunds behave after partial releases.

Multi-currency. The group has one base currency. Each invoice records its own currency, the exchange rate used, and the rate's date, so the ledger stays auditable. Verify in the sandbox which currencies Orders and Payouts accept. If NGN is not supported, use USD as the base currency in the demo and show naira as a display value.

8. Data model

- **users:** id, name, email, role.
- **groups:** id, name, traveler_id, status, approval_threshold, shared_cost_rule.
- **members:** id, group_id, user_id, contribution_expected.

- **order_lines:** id, group_id, member_id, item, quantity, budget.
- **contributions:** id, member_id, amount, currency, paypal_order_id, status.
- **release_requests:** id, group_id, amount, invoice_file, ai_report (JSON), status, paypal_payout_id.
- **approvals:** id, release_request_id, member_id, decision.
- **receipts:** id, group_id, file, ai_extraction (JSON).
- **allocations:** id, group_id, member_id, goods_total, shared_cost_share, net_owed_or_refund.
- **ledger_entries:** id, group_id, type, amount, paypal_ref, created_at.

Multi-currency fields. groups gains base_currency. release_requests and receipts each gain currency, original_amount, fx_rate, fx_rate_date and source_language. ledger_entries stores amounts in the base currency, with the original amount and currency beside them.

9. Architecture

- **Frontend:** Next.js (group dashboard, upload screens, ledger).
- **Backend:** NestJS REST API, PostgreSQL, file storage for invoices and receipts.
- **AI:** A multimodal model API called from the backend with a fixed JSON schema for extraction and a separate verification step.
- **PayPal:** Sandbox, with webhooks delivered to the backend.

10. Trust and reliability requirements

- Append-only ledger: entries are never edited, only reversed.
- Webhook handlers are idempotent and verify PayPal signatures.
- Payout actions require an explicit approval record.
- All uploads and AI results are retained for audit.

11. Demo scenario and success criteria

Three-minute demo.

1. The real trip story and the trust problem (20 seconds).
2. Five vendors fund the pot through PayPal.
3. Traveler uploads two invoices in different currencies (for example yuan and dirhams), one with a deliberate error. AI reads both, converts them to the base currency, and flags the error.
4. Corrected invoice is approved and a staged payout is released.

5. Final receipts go in. Each vendor sees their exact share and refund.

Success criteria.

- Full flow runs end to end in the sandbox with no manual database edits.
- AI catches the planted mismatch and extracts a messy receipt correctly.
- Every ledger line shows a PayPal reference.
- Submitted on Devpost before the deadline with video and write-up.

12. Timeline (Nov 6 to Nov 12)

Date	Focus
Nov 6	Sandbox setup, scaffold app, schema, group creation, PayPal contribution and webhook
Nov 7	Release request flow and staged payout
Nov 8	AI invoice extraction and verification
Nov 9	AI final allocation per vendor
Nov 10	Settlement, refunds, ledger view
Nov 11	Polish, seed demo data, record video, write Devpost description
Nov 12	Buffer and early submission

Cut order if behind: refunds and settlement first, then ledger polish. Never cut the AI check or the staged release.

13. Risks and open questions

- **PayPal fund holding.** The sandbox may constrain how platform-held funds and staged payouts behave. Verify early and adapt the flow.
- **Real-world payouts.** A Nigerian traveler receiving PayPal payouts has practical limits outside the sandbox. The write-up should say this plainly and describe how a production version would handle it.
- **Goods split rule.** Confirm whether vendors split goods by what each ordered or by money contributed. The default here is by what each ordered, with shared costs split proportionally.

- **AI accuracy on poor images.** Test with a blurry, mixed-language receipt by Nov 8, not Nov 11.
- **Teammate split.** Suggested: one person on PayPal and the money flow, one on AI and frontend.
- **Exchange rates.** Pick one rate source (a free rates API, or a fixed rate table for the demo) and store the rate with every entry. A live rate feed is not worth the time before Nov 12.